CS23532 – COMPUTER NETWORKS

EX 7: Flow Control at Data Link Layer (Sliding Window Protocol)

AIM :

To write a program to implement flow control at the data link layer using the SLIDING WINDOW PROTOCOL and to simulate the flow of frames from one node to another.

PROCEDURE :

Sender program

1. Input the window size from the user.
2. Input a text message from the user.
3. Consider 1 character per frame.
4. Create a frame with the following fields [Frame no., DATA].
5. Send the frames. Print the output on the screen and save it in a file called Sender_Buffer.
6. Wait for the acknowledgement from the receiver (a delay is induced in the program).
7. Read the file called Receiver_Buffer.
8. Check the ACK field for the acknowledgement number.
9. If the acknowledgement number is as expected, send the new set of frames accordingly (overwrite the Sender_Buffer file with the new frames). Else, if a NACK is received, resend the frames accordingly (overwrite the Sender_Buffer with the old frames).

Receiver program

1. Read the file called Sender_Buffer.
2. Check the Frame no.
3. If the Frame no. is as expected, write the appropriate ACK no. in the Receiver_Buffer file. Else, write the NACK no. in the Receiver_Buffer file.

PROGRAM:

sender.py

```
# Sliding Window Protocol - SENDER
import time

window = int(input("Enter the window size: "))
message = input("Enter the message (binary data): ")
error_frame = int(input("Enter the frame number to introduce an error (-1 for
no error): "))
total = len(message)
base = 0                                    # first unacknowledged frame
open("Receiver_Buffer", "w").close()

while base < total:
    end = min(base + window, total)
    with open("Sender_Buffer", "w") as f:  # overwrite buffer with new window
        for i in range(base, end):
            if i == error_frame:             # introduce error (first
transmission only)
                f.write(f"{i + 2},{message[i]}\n")   # wrong Frame no. in the
buffer
                print(f"Sent Frame {i} : DATA = {message[i]}  (error
introduced: Frame no. changed to {i + 2})")
                error_frame = -1
            else:
                f.write(f"{i},{message[i]}\n")  # frame = [Frame no., DATA]
                print(f"Sent Frame {i} : DATA = {message[i]}")
    print("Waiting for acknowledgement...")
    time.sleep(2)                           # induced delay
    reply = ""
    while not reply:
        time.sleep(1)
        reply = open("Receiver_Buffer").read().strip()
    open("Receiver_Buffer", "w").close()
    kind, num = reply.split()
    num = int(num)
    if kind == "ACK":
        print(f"Received ACK {num} (as expected) - sending next window\n")
    else:
        print(f"Received NACK {num} - resending frames from {num}\n")
    base = num

with open("Sender_Buffer", "w") as f:
    f.write("END\n")
print("All frames sent and acknowledged.")
```

receiver.py

```
# Sliding Window Protocol - RECEIVER
import os, time

expected = 0                                # next frame number expected
received = ""
while True:
    data = open("Sender_Buffer").read().strip() if
os.path.exists("Sender Buffer") else ""
    if not data:
        time.sleep(0.5)
        continue
    if data == "END":
```

```
        break
    time.sleep(1.5)                          # frame transmission delay
    lines = open("Sender_Buffer").read().split()
    open("Sender_Buffer", "w").close()
    error = False
    for line in lines:
        num, bit = line.split(",")
        num = int(num)
        if num == expected:
            print(f"Received Frame {num} : DATA = {bit}  -> accepted")
            received += bit
            expected += 1
        else:
            print(f"Received Frame {num} : expected Frame {expected} ->
discarded")
            error = True
            break
    with open("Receiver_Buffer", "w") as f:
        f.write(f"{'NACK' if error else 'ACK'} {expected}\n")
    print(f"{'NACK' if error else 'ACK'} {expected} written to
Receiver_Buffer\n")

print("Message received:", received)
```

OUTPUT:

Input

Window size = 3, message (sample binary data) = 10110010. Test 1: no error (-1). Test 2: error introduced in Frame 3.

Test 1 – Sender output (no error)

```
Enter the window size: 3
Enter the message (binary data): 10110010
Enter the frame number to introduce an error (-1 for no error): -1
Sent Frame 0 : DATA = 1
Sent Frame 1 : DATA = 0
Sent Frame 2 : DATA = 1
Waiting for acknowledgement...
Received ACK 3 (as expected) - sending next window

Sent Frame 3 : DATA = 1
Sent Frame 4 : DATA = 0
Sent Frame 5 : DATA = 0
Waiting for acknowledgement...
Received ACK 6 (as expected) - sending next window

Sent Frame 6 : DATA = 1
Sent Frame 7 : DATA = 0
Waiting for acknowledgement...
Received ACK 8 (as expected) - sending next window

All frames sent and acknowledged.
```

Test 1 – Receiver output (no error)

```
Received Frame 0 : DATA = 1  -> accepted
Received Frame 1 : DATA = 0  -> accepted
Received Frame 2 : DATA = 1  -> accepted
ACK 3 written to Receiver_Buffer

Received Frame 3 : DATA = 1  -> accepted
Received Frame 4 : DATA = 0  -> accepted
Received Frame 5 : DATA = 0  -> accepted
ACK 6 written to Receiver_Buffer

Received Frame 6 : DATA = 1  -> accepted
Received Frame 7 : DATA = 0  -> accepted
ACK 8 written to Receiver_Buffer

Message received: 10110010
```

Test 2 – Sender output (error introduced in Frame 3)

```
Enter the window size: 3
Enter the message (binary data): 10110010
Enter the frame number to introduce an error (-1 for no error): 3
Sent Frame 0 : DATA = 1
Sent Frame 1 : DATA = 0
Sent Frame 2 : DATA = 1
Waiting for acknowledgement...
Received ACK 3 (as expected) - sending next window
```

```
Sent Frame 3 : DATA = 1  (error introduced: Frame no. changed to 5)
Sent Frame 4 : DATA = 0
Sent Frame 5 : DATA = 0
Waiting for acknowledgement...
Received NACK 3 - resending frames from 3

Sent Frame 3 : DATA = 1
Sent Frame 4 : DATA = 0
Sent Frame 5 : DATA = 0
Waiting for acknowledgement...
Received ACK 6 (as expected) - sending next window

Sent Frame 6 : DATA = 1
Sent Frame 7 : DATA = 0
Waiting for acknowledgement...
Received ACK 8 (as expected) - sending next window

All frames sent and acknowledged.
```

Test 2 – Receiver output (error introduced in Frame 3)

```
Received Frame 0 : DATA = 1  -> accepted
Received Frame 1 : DATA = 0  -> accepted
Received Frame 2 : DATA = 1  -> accepted
ACK 3 written to Receiver_Buffer

Received Frame 5 : expected Frame 3 -> discarded
NACK 3 written to Receiver_Buffer

Received Frame 3 : DATA = 1  -> accepted
Received Frame 4 : DATA = 0  -> accepted
Received Frame 5 : DATA = 0  -> accepted
ACK 6 written to Receiver_Buffer

Received Frame 6 : DATA = 1  -> accepted
Received Frame 7 : DATA = 0  -> accepted
ACK 8 written to Receiver_Buffer

Message received: 10110010
```

RESULT:

Thus, a program to implement flow control at the data link layer using the sliding window protocol was written and executed. Frames were sent window by window and acknowledged by the receiver, and when an error was induced the receiver sent a NACK and the sender retransmitted the frames, so the complete message was received correctly.